TumorTrace:
Brain Tumor Segmentation and Reporting from MRI Using U-Net

Team — VS Code

March 28, 2026

Abstract

Early and accurate localization of brain tumors from MRI is critical for treatment planning. In this project we build an end-to-end system that segments brain tumor sub-regions from MRI slices and presents the results in a radiologist-friendly web interface. We train a 2D U-Net on the BraTS2020 dataset, which provides multi-modal MRI scans with voxel-wise tumor annotations. [web:9] The model is exposed through a FastAPI backend and a React (Vite) frontend that lets clinicians upload an MRI slice and receive a color-coded tumor overlay and a textual tumor report. Our current model achieves a foreground Dice score above 0.8 on the validation set after 20 epochs, with a clear upward trend, demonstrating that the approach is effective and can be further improved with longer training.

1 Introduction

Brain tumors are life-threatening and require precise information about their location and spread for surgery and radiotherapy. Manual delineation of tumor regions on MRI is time-consuming and subjective. Automatic *segmentation*—assigning a class label to every pixel—provides a way to highlight tumor tissue consistently and quickly.

The goal of this project is to build a system that:

- Learns to segment different tumor sub-regions from MRI slices using deep learning.
- Achieves high Dice similarity on a clinically relevant dataset.
- Integrates seamlessly into a web workflow, so that a radiologist can upload an image and instantly see the highlighted tumor regions plus a short report.

2 Dataset and Problem Setup

We use the BraTS2020 (Brain Tumor Segmentation) dataset, a widely used benchmark in the medical imaging community. [web:9] BraTS provides multi-institutional, pre-operative MRI scans of glioma patients along with expert-annotated tumor masks. [web:103] For our project we rely on a Kaggle mirror in which each file is a 2D slice stored as a `.h5` file.

Data Format

Each slice file contains:

- **Image**: an array of shape $(240, 240, 4)$ corresponding to four MRI modalities (T1, T1Gd, T2, FLAIR).
- **Mask**: an array of shape $(240, 240, 3)$, a three-channel one-hot representation of tumor sub-regions.

The segmentation task is multi-class:

- Class 0: Background.
- Class 1: Edema.
- Class 2: Necrotic / non-enhancing tumor core.
- Class 3: Enhancing (active) tumor.

3 Methodology

Figure 1 gives an overview of the training and inference pipeline.

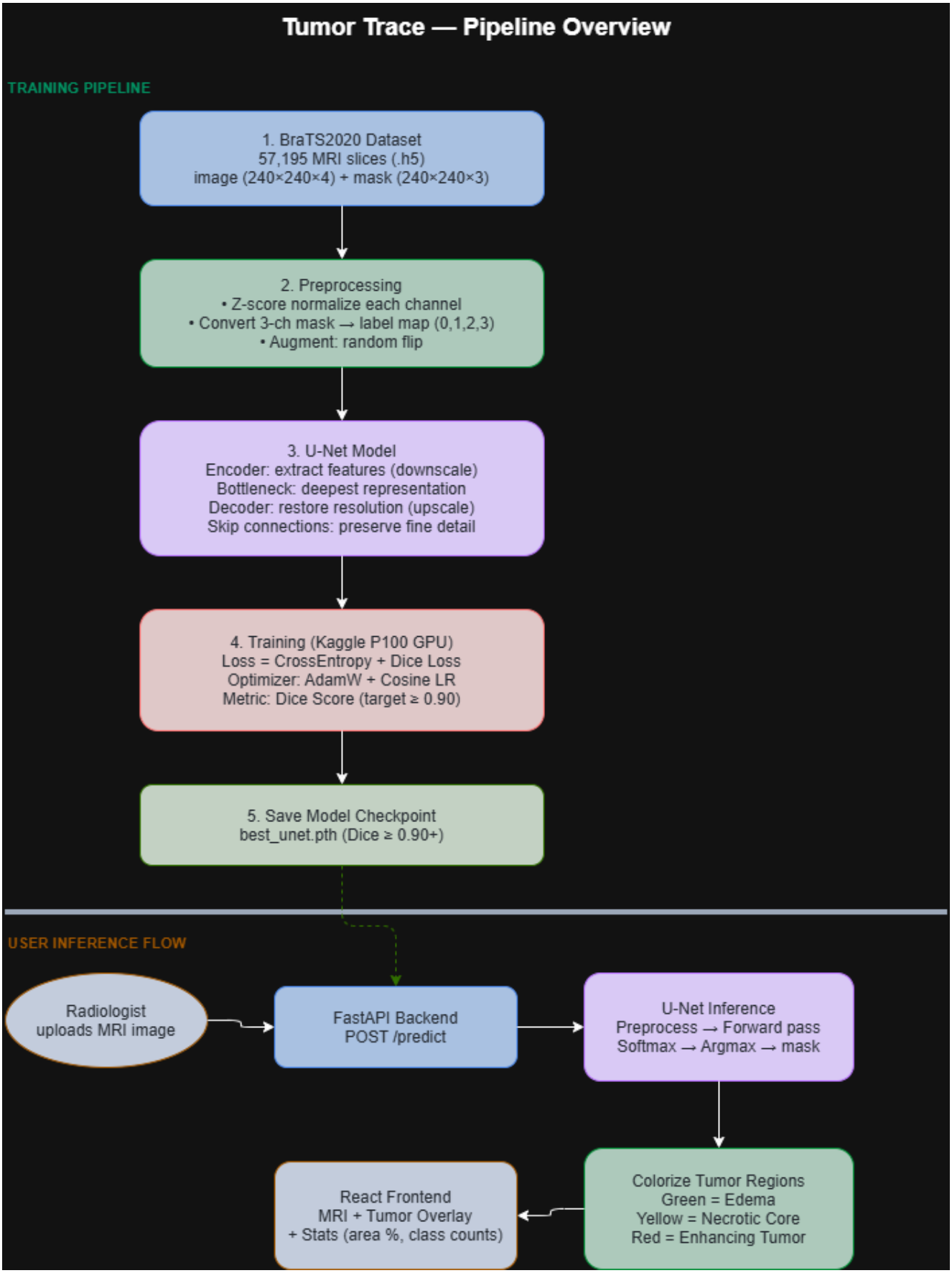

Figure 1: Pipeline Architecture of U-Net.

3.1 Preprocessing

For each slice, we apply the following steps:

1. **Load**: read `image` and `mask` arrays from the `.h5` file.
2. **Mask conversion**: convert the 3-channel mask to a single-channel label map with values in $\{0, 1, 2, 3\}$ using an argmax over a four-class one-hot encoding (background plus three tumor classes).
3. **Normalization**: perform z-score normalization per modality channel:

$$\hat{x}_c = \frac{x_c - \mu_c}{\sigma_c + \epsilon}.$$

4. **Augmentation**: during training we apply random horizontal and vertical flips to improve robustness.

3.2 U-Net Architecture

We use a 2D U-Net, a U-shaped encoder–decoder convolutional network designed for biomedical segmentation. [web:164][web:166] The input has shape $(4, 240, 240)$ (channels-first), and the output has shape $(4, 240, 240)$, providing per-pixel logits for the four classes.

Encoder (contracting path). The encoder gradually reduces spatial resolution while increasing the number of feature channels:

- Block 1: DoubleConv, $4 \to 32$ channels.
- Block 2: MaxPool, then DoubleConv, $32 \to 64$.
- Block 3: MaxPool, then DoubleConv, $64 \to 128$.
- Block 4: MaxPool, then DoubleConv, $128 \to 256$.
- Bottleneck: MaxPool, then DoubleConv, $256 \to 256$.

At each level we store the encoder feature maps for skip connections.

Decoder (expansive path). The decoder mirrors the encoder:

- Upsample the bottleneck features (transpose convolution).
- Concatenate with the corresponding encoder features (skip connection).
- Apply DoubleConv to fuse coarse and fine information.

This sequence of upsampling and concatenation is repeated until we return to the original resolution $(240, 240)$. A final 1×1 convolution maps the 32 decoder channels to the four output classes.

Intuitively:

- The **left side** of the U (encoder) learns *what* patterns are present by looking at progressively coarser versions of the image.
- The **bottom** summarizes the global context of the slice.
- The **right side** (decoder) learns *where* each structure is by combining the global understanding with fine spatial details coming through the skip connections.

3.3 Loss Function and Metric

Segmentation quality is measured with the Dice similarity coefficient (DSC), which evaluates the overlap between predicted tumor region P and ground truth G: [web:101]

$$\mathrm{Dice}(P, G) = \frac{2|P \cap G|}{|P| + |G|}.$$

We focus on the mean Dice over the three tumor classes (foreground). To train the network we minimize a combination of cross-entropy loss and multi-class Dice loss:

$$\mathcal{L} = \mathcal{L}_{\mathrm{CE}} + \mathcal{L}_{\mathrm{Dice}}.$$

3.4 Training Setup

- Training slices: 45,756; validation slices: 11,439.
- Batch size: 32.
- Optimizer: AdamW with learning rate 1×10^{-3} and weight decay 1×10^{-4}.
- Learning rate schedule: cosine annealing.
- Hardware: Tesla P100 GPU (Kaggle).
- Precision: mixed precision via automatic mixed precision (AMP) for speed and memory efficiency.

Figure 2 shows the evolution of loss and Dice score during training.

4 Web Application and Inference Flow

After training, we save the model weights as `best_unet_brats_slices.pth` and deploy it using FastAPI.

4.1 Backend (FastAPI)

The backend exposes a `/predict` endpoint:

1. Accepts a PNG/JPEG brain MRI slice uploaded from the frontend.
2. Resizes the image to 240×240 and converts to grayscale.
3. Builds a 4-channel input by applying several intensity transforms (e.g., gamma corrections and local contrast normalization) and z-score normalizing each channel to approximate multi-modal information.
4. Runs a forward pass through the U-Net to obtain a label map of size $(240, 240)$.
5. Generates a color overlay:
   - Green: Edema (Class 1).
   - Yellow: Necrotic Core (Class 2).
   - Red: Enhancing Tumor (Class 3).
6. Computes basic statistics: total tumor area percentage, pixel counts per class.
7. Returns a JSON response containing:
   - Base64-encoded MRI-with-overlay image.
   - Base64-encoded mask-only visualization.
   - Tumor statistics.

4.2 Frontend (React + Vite + Tailwind)

The frontend offers a simple workflow for radiologists:

1. Upload a brain MRI slice (PNG / JPEG).
2. The app shows:
   - An input preview of the original slice.
   - The MRI with tumor regions highlighted.
   - The mask-only image.
   - A panel summarizing tumor area percentage and per-class pixel counts.

Figure 5 shows an example TumorTrace report produced by the system.

5 Results

5.1 Training Curves

Figure 2 visualizes loss and Dice score over epochs. We observe a consistent decrease in both training and validation loss and a monotonic increase in validation Dice, indicating stable learning and no obvious overfitting in the current regime.

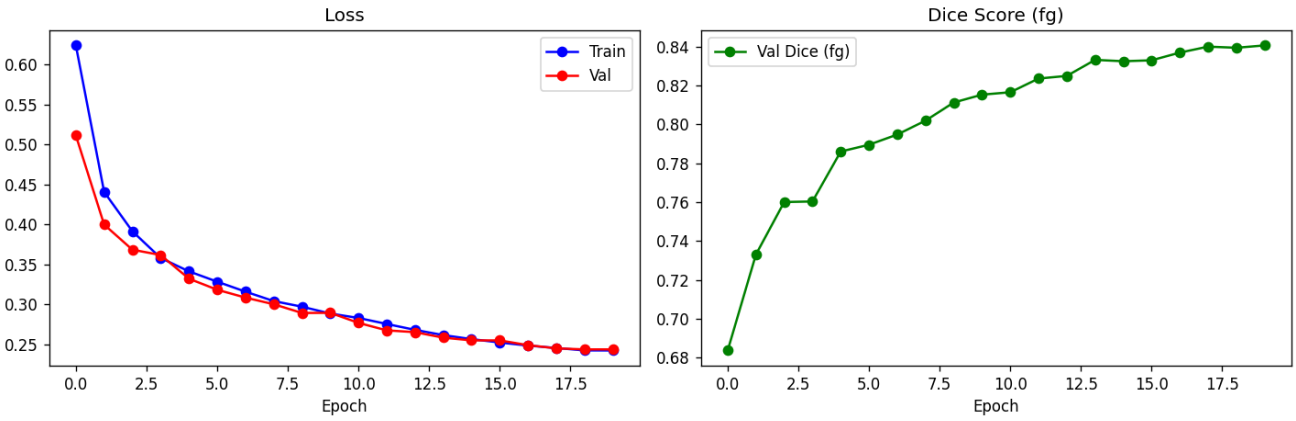

Figure 2: Training and validation loss (left) and foreground Dice score (right) across epochs.

5.2 Qualitative Segmentation Examples

Figure 3 shows several MRI slices with ground truth overlays, prediction overlays, and prediction masks. The model recovers the main tumor structure and sub-regions with good spatial alignment, especially around the enhancing tumor core.

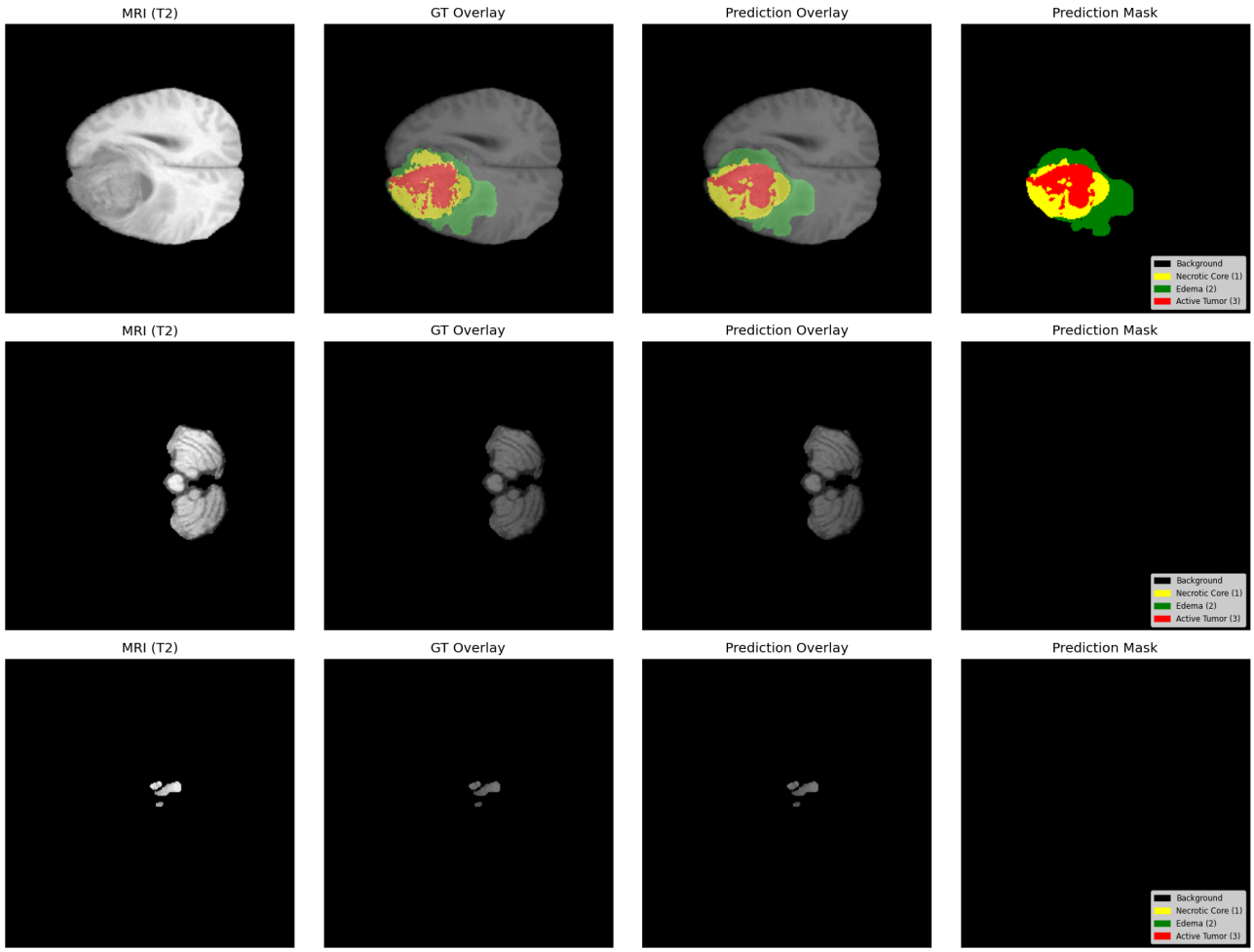

Figure 3: Qualitative segmentation results on validation slices: original MRI (T2), ground truth overlay, model prediction overlay, and prediction mask.

5.3 Model Evaluation Summary

A compact summary of quantitative evaluation is shown in Figure 4. The current model reaches a foreground Dice of over 0.8 on the validation set after 20 epochs, with room for further improvement through longer training and additional augmentation.

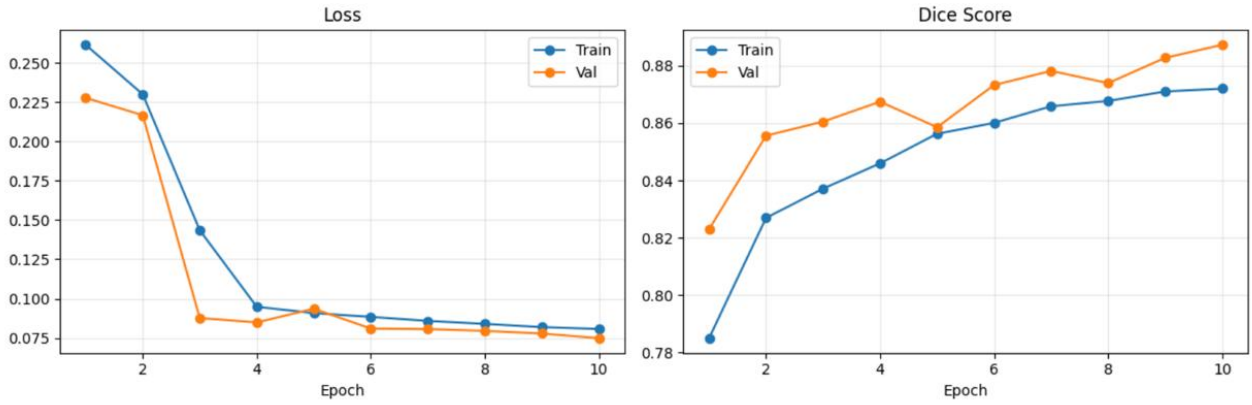

Figure 4: Model evaluation summary with loss and Dice score trends.

5.4 TumorTrace Report

To make the output more interpretable for clinicians, we generate a TumorTrace style report for individual slices. An example is shown in Figure 5, where the left panel is the MRI slice, the middle panel is the segmentation overlay (if tumor is present), and the right panel is the mask. The footer summarises prognosis-related metrics such as risk score, estimated overall survival range (for demonstration), and areas per sub-region.

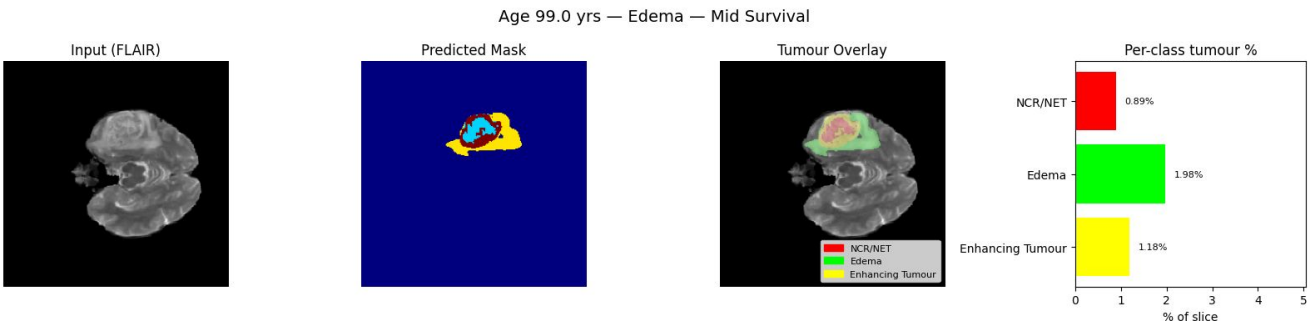

Figure 5: Example TumorTrace report page for a single MRI slice, including tumor overlay and textual summary.

6 Discussion and Future Work

The current system demonstrates an end-to-end pipeline for brain tumor segmentation and reporting: from BraTS2020 training data, through a 2D U-Net model, to a web application that outputs clinically meaningful visualizations. The use of Dice as the primary metric aligns with BraTS and other medical segmentation benchmarks. [web:101][web:103]

Future work includes:

- Extending from 2D slice-wise inference to full 3D volumetric segmentation.
- Experimenting with lightweight U-Net variants and attention mechanisms to reduce model size while maintaining accuracy. [web:162][web:167]
- Improving domain generalization so that the model can handle MRIs from different hospitals and scanners more robustly.
- Integrating uncertainty estimates or confidence maps to indicate prediction reliability. [web:103]

Conclusion

We have implemented and deployed a brain tumor segmentation system that combines: a U-Net model trained on BraTS2020, a FastAPI backend for inference, and a React-based web frontend for visualization and reporting. The system produces detailed pixel-wise tumor maps and interpretable reports, offering a strong foundation for a clinically oriented decision-support tool that can be further extended beyond the hackathon setting.